

DLMO User Guide on Discrimination and Detection Tasks

Version 1.0.0

<https://github.com/DIDSR/DLMO>

June 12, 2026

Contents

1	Summary	3
2	Requirements	3
3	Usage	4
3.1	Demo 1: Object generation using DDPM	4
3.2	Demo 2: Signal Insertion	6
3.3	Demo 3: MR acquisition and rSOS reconstruction	8
3.4	Demo 4: DLMO training	11
3.5	Demo 5: DLMO testing output	13
3.6	Demo 6: Statistical analysis	15
4	References	18
5	Contact	18

1 Summary

Deep Learning Model Observer (DLMO) is a computational assessment method for comparing the task-based performance of brain MR images reconstructed using a new method from undersampled data with that of images reconstructed using a reference method from fully sampled data. It outputs discrimination- or detection-based performance metrics—percent correct (PC) or area under the receiver operating characteristic curve (AUROC)—for both methods, enabling direct performance comparison.

This user manual includes a use-case demo of the DLMO-based assessment on a discrimination task as a surrogate for MR acceleration, as detailed in our [DLMO paper](#) [1]. The implementation simulates multi-coil, sensitivity-encoded parallel MR imaging at varying acceleration factors, using fully sampled root of sum of squares (rSOS) reconstruction as the reference and U-Net post-processing of undersampled reconstructions as the test method.

The signal content can be adapted to a signal-present versus signal-absent configuration for detection tasks, as described in the [CNN-IO paper](#) [2]. The same DLMO approach can also be applied to other acquisition settings and reconstruction methods.

2 Requirements

To run the DLMO code, begin by cloning the following GitHub repository:

```
1 git clone https://github.com/DIDSR/DLMO.git
```

The trained models uploaded to this repository total approximately 2.17 GB. In case of bandwidth issues or if you only need to use the code, you can enable GitHub's smudging option to download the models as pointers instead.

```
1 # to avoid LFS quota issue and to download this repo without the trained models
2 GIT_LFS_SKIP_SMUDGE=1 git clone https://github.com/DIDSR/DLMO.git
3 # in case you need to fully download/pull all the models in this repo
4 git lfs pull
```

Denoising Diffusion Probabilistic Models (DDPM)–based object creation and deep learning–based training and inference environments have been separated into `ddpm.yml` and `dlmo.yml`. You may combine them at your own risk, or create separate environments for each of them as:

```
1 # to create MR objects
2 conda env create --name ddpm --file requirements/ddpm.yml
3 # to run the dlmo training/testing
4 conda env create --name dlmo --file requirements/dlmo.yml
```

Our `ddpm` environment is based on OpenAI's DDPM code, which can be built separately following the instructions in <https://github.com/openai/improved-diffusion>.

Similarly, our `dlmo` environment is based on Horovod's NCCL-based distributed training across multiple GPUs. However, you **DO NOT NEED** to install Horovod to run the demos provided in

this repository. DLMO training and testing can run on a single GPU without importing Horovod. Accordingly, the Horovod installation line has been commented out in the `dlmo.yaml` file.

If you wish to perform multigpu-based training for your AI reconstruction or DLMO training, follow the instructions provided in <https://github.com/NVIDIA/ncc1> and https://horovod.readthedocs.io/en/latest/install_include.html.

3 Usage

Example codes below demonstrate how to run the DLMO assessment methodology for a discrimination task, serving as a surrogate for image resolution across reconstruction algorithms as a function of MR acceleration.

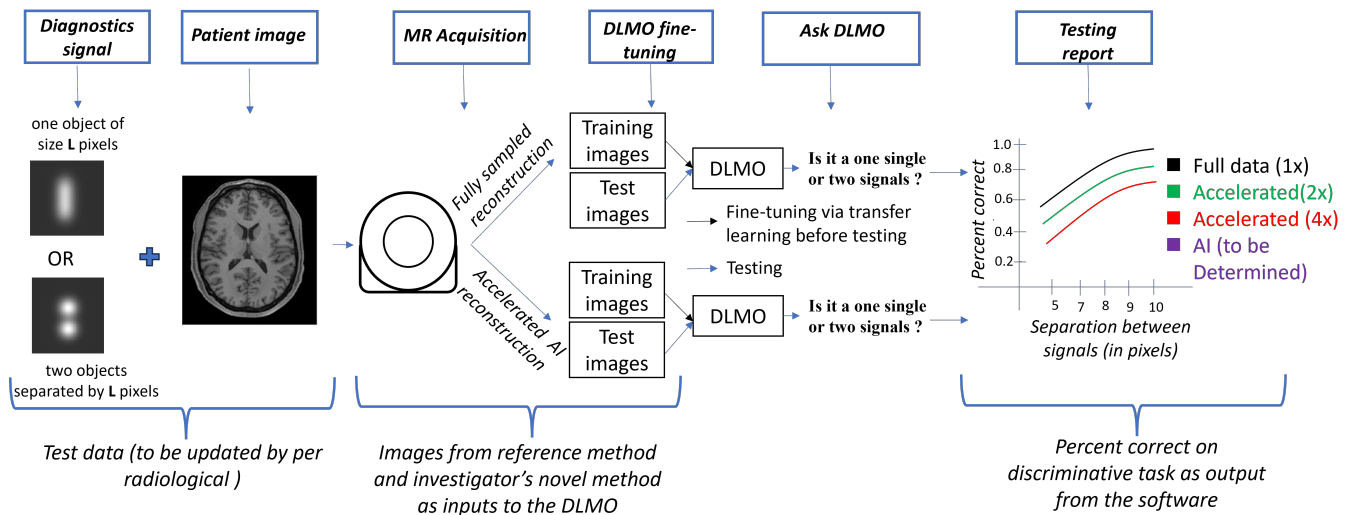


Figure 1: An in silico imaging trial using DLMO to evaluate AI-based accelerated MRI reconstruction on discrimination task

3.1 Demo 1: Object generation using DDPM

The code below uses a trained Denoising Diffusion Probabilistic Model (DDPM) [3] to generate a large batch of 2D MR image samples that are saved as a numpy array. These are also referred to as Stochastic Object Models (SOMs) in the rest of the implementation.

```
1 conda activate ddpm
2 cd src/demo1
3 # Python file and its input and outputs -----
4 PY_FILE=scripts/image_sample_newdataset2_centercrop.py
5 MODEL_PATH=trained_DDPM_model/ema_0.9999_1100000.pt
6 OUTPUT_FLD="test_out_400k"
7
8 # Important ddpm parameters -----
9 dSTEP=1000
10 nSAMPLES=400000
11 bSIZE=1
```

```

12
13 # CMD_ARGUMENTS aligning with how we trained the DDPM model -----
14 MODEL_FLAGS="--image_size 384 --attention_resolutions 32,16,8 --num_channels 128 --
    num_head_channels 64 --num_res_blocks 2 --resblock_updown True --use_scale_shift_norm
    True --learn_sigma True"
15 DIFFUSION_FLAGS="--diffusion_steps ${dSTEP} --noise_schedule cosine "
16 sample_FLAGS="--save_dir ${OUTPUT_FLD}/
    HCP_brain_384x384_cropped_260x311_step1100k_ema_samples/ --num_samples ${nSAMPLES} --
    batch_size ${bSIZE}"
17
18 time python ${PY_FILE} --model_path ${MODEL_PATH} $MODEL_FLAGS ${DIFFUSION_FLAGS} ${
    sample_FLAGS}

```

The generated .npz file will be used as input for signal insertion and MR reconstruction. **Additional information:**

1. The provided pre-trained DDPM model [3] was trained using the Human Connectome Project (HCP) Young Adult dataset [4]:
 - a. This dataset consists of 1,113 subjects scanned on a customized Siemens 3T MRI system.
 - b. From each patient, 10 axial slices within their Cerebrospinal fluid (CSF) regions were extracted to train the DDPM model
2. The number of diffusion steps is an important parameter when running this ddpm-based MRI data generation code.

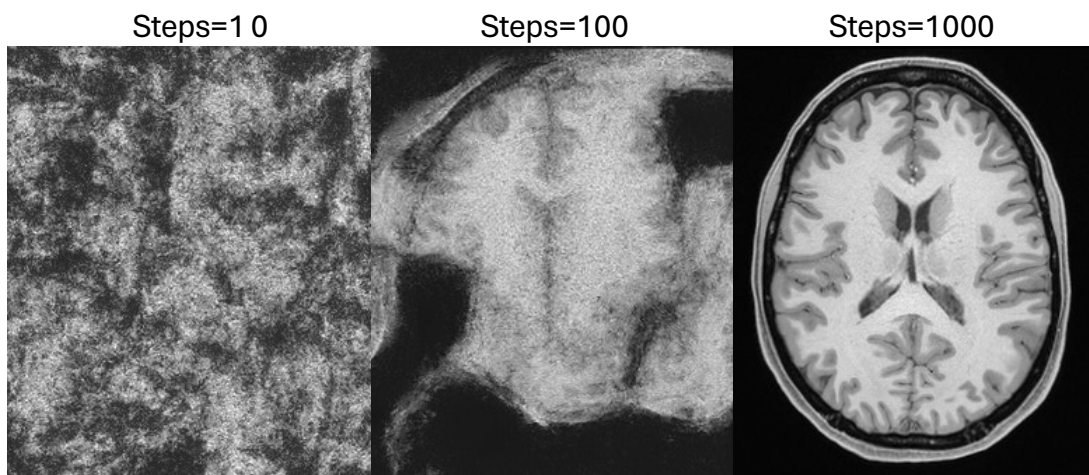


Figure 2: MR images for different diffusion steps

3. Appropriately validate the generated data using metrics relevant to your clinical application. Fig. 3 lists image statistics corresponding to the mean, standard deviation, white matter area-to-intracranial area ratio, and gray matter area-to-intracranial area ratio, computed using 1,000 real and 1,000 DDPM-generated MR images at each step.
4. Approximately 433,330 images were generated, including 336,000 for DLMO training, 80,000 for UNet training, 16,000 for image fidelity and percent correct (or AUC) analyses, and 1,330

Image statistics	Mean	Std	R_{white}	R_{gray}
Real image	0.3039 ± 0.0354	0.2328 ± 0.0201	0.3124 ± 0.1089	0.4160 ± 0.0816
1,000 steps	0.3278 ± 0.0565	0.2496 ± 0.0319	0.3250 ± 0.0930	0.3901 ± 0.0690
100 steps	0.3719 ± 0.0854	0.2749 ± 0.0523	0.3933 ± 0.0987	0.3353 ± 0.0826

Figure 3: Image statistics for different diffusion steps

for the 2AFC validation study of DLMO.

3.2 Demo 2: Signal Insertion

This implementation inserts doublet (or two dots) and singlet (or single dot) signals into DDPM-generated MR images (previously detailed in section 3.1). The singlet and doublet signals for different signal lengths are determined using the 2AFC-based discrimination table shown in fig. 6. Consequently, the signals are defined by acceleration factor, signal contrast, and signal length (in pixels). The resulting images with inserted signals are saved in HDF5 format.

Command-line input options:

```

1 acceleration (int) : Acceleration factor for sparse sampling (2, 4, 6, or 8).
2 contrast (float) : Signal amplitude/contrast value.
3 signal_lengths (str) : Comma-separated signal separation lengths, e.g. "4,5,6,7,8".
4 object_hdf5_path (str, optional): Path to the DDPM-generated objects from demo 1.
```

Command-line code:

```

1 python signal_insertion_test.py [acceleration factor] [contrast] [signal_lengths] [
   object_npz_path]
```

Example: A demo run with the acceleration factor set to 4 and the signal length and intensity values corresponding to the 7th row of the 2-AFC table in fig. 6 (this row-based signal lengths and intensity values are also used for testing in our DLMO paper [1]):

```

1 python signal_insertion_test.py 4 0.7 '4,5,6,7,8'
```

Outputs: Each output HDF5 file contains the following datasets: H_s (singlet images), H_d (doublet images), and L_list (signal lengths).

Note that the limiting conditions for different acceleration factor using iFFT-based reconstruction was determined across a series of 2AFC studies performed by a trained non-physician reader as shown in fig. 6. Each cell corresponds to one 2AFC study defined by a specific combination of signal intensity and signal length. Colored cells indicate conditions under which the reader achieved 100% accuracy on the Rayleigh discrimination task. Note that whenever 100% accuracy was achieved at a higher acceleration factor, the same signal condition also yielded 100% accuracy at lower acceleration factors. For example, a signal with intensity 1.3 and length 8 mm resulted in perfect accuracy at acceleration factors of $8\times$, then the same condition is expected to yield 100% accuracy is expected at $1\times$, and $4\times$ accelerations. Also, for a given signal intensity, note that

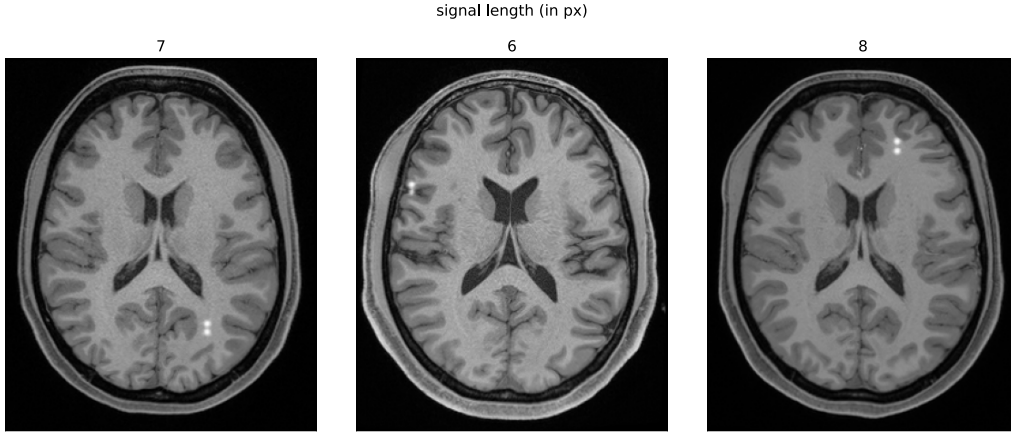


Figure 4: Example MR images with the doublet signal corresponding to the demo run.

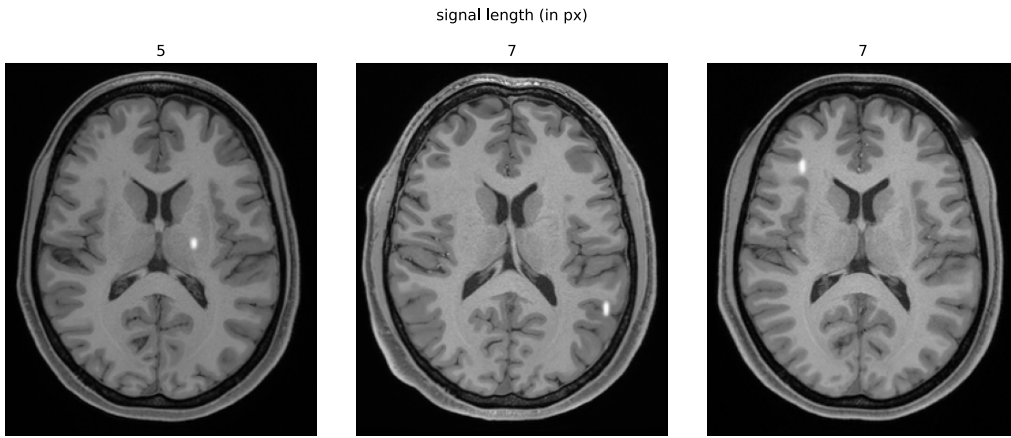


Figure 5: Example MR images with the singlet signal corresponding to the demo run.

whenever 100% accuracy was achieved at a particular signal length, all longer signal lengths yielded 100% accuracy as well. For example, for the acceleration factor $1\times$, 100% accuracy was observed for a signal intensity of 1.3 at a length of 4 mm. The same held true for all lengths greater than 4 mm at the same 1.3 intensity for $1\times$. Here, $1\text{px} = 1\text{mm}$.

The shaded boxes in fig. 6 indicate combinations of intensity and signal length used to generate testing images that encompass limiting conditions (for acceleration factors 4 and 8), to evaluate whether AI-based reconstruction provides better discriminatory capability than conventional iFFT-based reconstruction.

The relevance of this 2AFC table can also be inferred from its close alignment with the smallest signal length (4 mm) and the lowest signal intensity (0.3), which are similar to those observed in non-specific white lesion (NSWL) distributions from the fastMRI+ dataset [5]. For this analysis, we predominantly included NSWLs with height and width $< 20\text{ px}$ from the fastMRI+ dataset.

From these plots, we observe that NSWLs can be as small as 4–5 px in both width and height, with lower-end intensity values ranging from approximately 50/256 to 75/256. These limiting sizes and intensity values are similar to those obtained in our 2AFC study when MR acquisition is performed at a fully sampled rate.

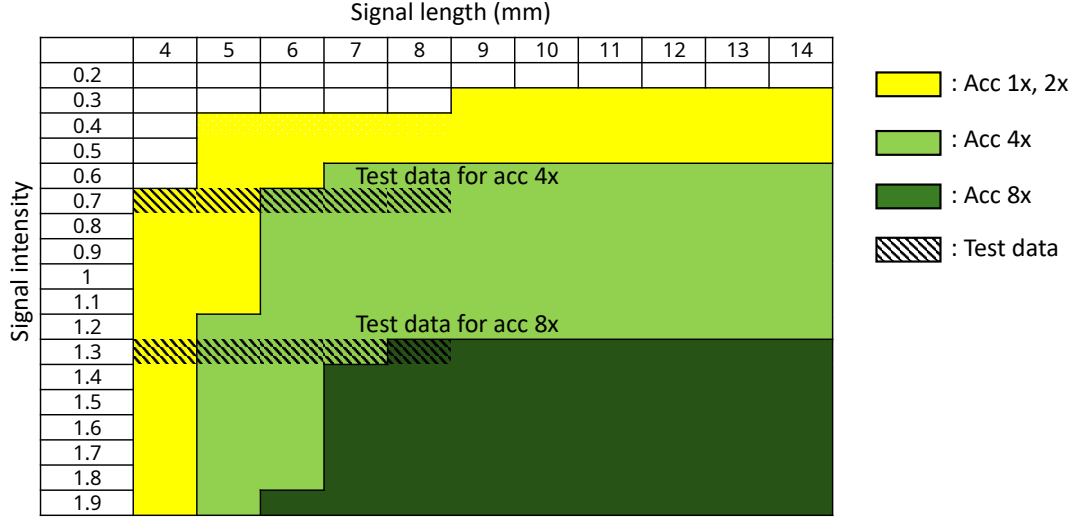


Figure 6: 2AFC-based limiting condition for iFFT reconstruction at different acceleration factor

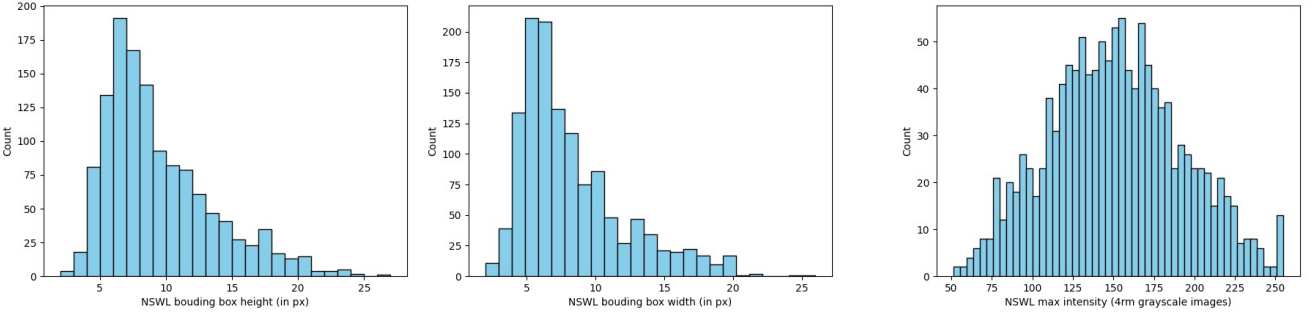


Figure 7: Distribution of bounding box height, width, and signal intensity corresponding to non-specific white lesions (NSWLs) in the fastMRI+ dataset.

We want our imaging system and its reconstruction method to perform as well as possible under limiting conditions—i.e., to enable detection and discrimination of the most challenging combinations of signal length and signal intensity. Accordingly, we assess whether discriminatory performance using AI-based reconstruction at an acceleration factor of 4 matches that of iFFT-based reconstruction at an acceleration factor of 1. Similar comparisons are made across other acceleration factors.

These comparisons are first performed along the boundary for a given acceleration factor (i.e., at 0.7 when comparing $4\times$ with $1\times$, and at 1.3 when comparing $8\times$ with $1\times$, corresponding to the shaded boxes in Fig. 6). If the accelerated reconstruction yields performance comparable to the fully sampled case, the signal intensity is then sequentially decreased (e.g., to 0.6 for the $4\times$ vs. $1\times$ comparison).

3.3 Demo 3: MR acquisition and rSOS reconstruction

This implementation shows forward projection and reconstruction of DDPM-generated objects (SOMs) using the rSOS method to create a test dataset. The reconstructed images from this

implementation are saved in HDF5 format.

```
1 acceleration_factor (int): Acceleration factor for sparse sampling (2, 4, 6, or 8).
2 object_hdf5_path (str, optional): Path to the DDPM-generated SOMs with added signals from
  demo 2.
```

Usage:

```
1 python rsos_ddpm_test.py [acceleration_factor] [object_hdf5_path]
```

An **Example** run with acceleration factor set as 4:

```
1 python rsos_ddpm_test.py 4
```

Input files to this demo run are .hdf5 files obtained from Demo 2. The reconstructed images are saved in HDF5 format. Each HDF5 file contains the following datasets: H_s for singlet image reconstructions, H_d for doublet image reconstructions, and L_list for the signal lengths corresponding to each reconstructed image.

MR SOMs with the doublet and singlet signals – as shown in figs. 4 and 5 are inputs, $f(r)$, to this code.

The FFT data collected at each coil is modeled as:

$$g_i = \Phi \mathcal{F} S_i f(r) + n_i,$$

where n_i denotes zero-mean Gaussian noise at each coil with a standard deviation set as 15. Our forward model uses 8 coils (S_i) and a Poisson disk-based subsampling pattern (Φ), as shown below:

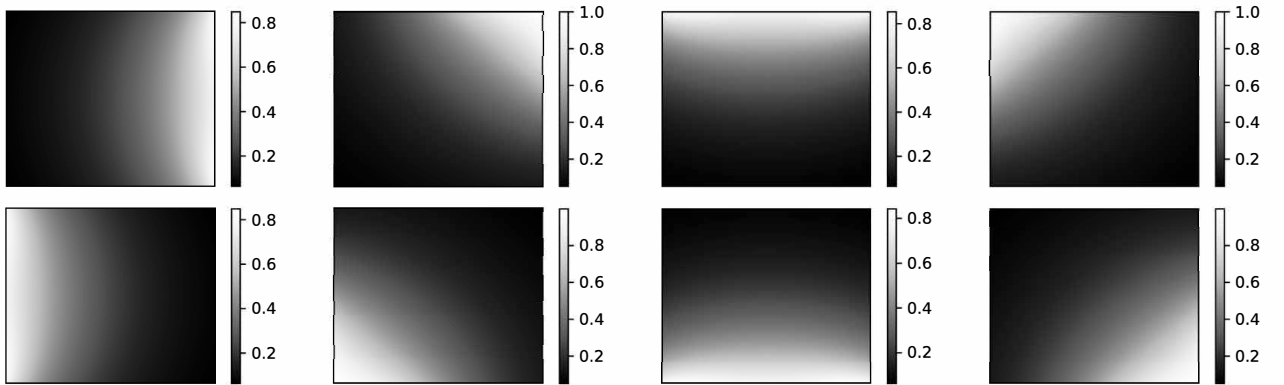


Figure 8: Eight-coiled sensitivity maps.

Then reconstruction at each coil is combined using iFFT in the following manner:

$$\hat{f}_i = \mathcal{F}^{-1} g_i,$$

$$\hat{f}_{\text{rSOS}} = \sqrt{\sum_{i=1}^{N_c} |\hat{f}_i|^2}.$$

Both the accelerated and fully sampled rSOS reconstructions are saved in the final HDF5 files using this implementation.

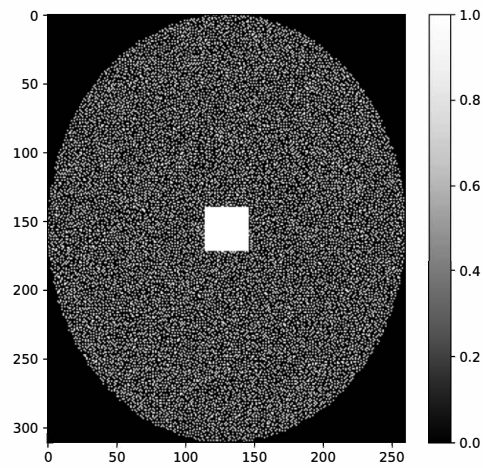


Figure 9: Poisson disk-based subsampling pattern for 4x.

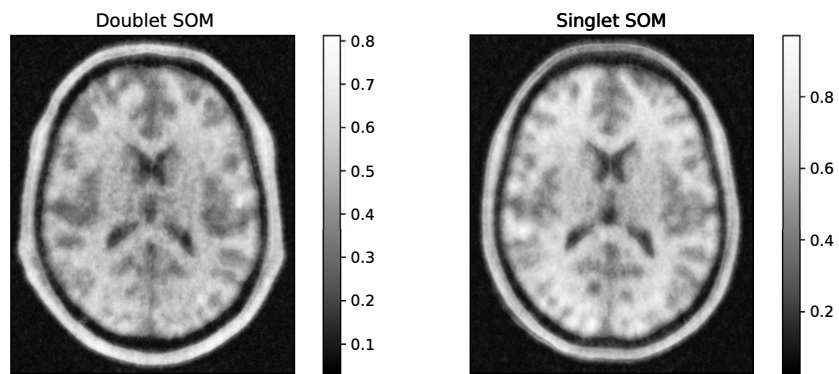


Figure 10: rSOS-based reconstruction examples with signals at an acceleration factor of 4.

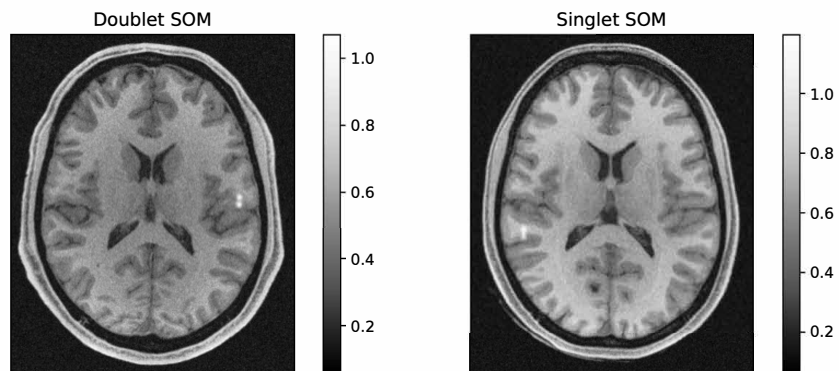


Figure 11: rSOS-based reconstruction examples with signals at an acceleration factor of 1.

3.4 Demo 4: DLMO training

The deep learning–based model observer (DLMO) training code provided here supports distributed training using Horovod, as well as serial training without Horovod (using 1 GPU). It also accommodates various configurations through command-line arguments.

Note that in our DLMO paper [1], the total number of training MR images for both the base model (i.e., acceleration = 1) and the refined DLMOs for each acceleration factor (i.e., 4× and 8×) was 168,000. To re-obtain the backbone-trained model (for acc = 1) and the re-tuned model (for acc > 1) provided in this repository and discussed in our DLMO paper, you will need to use training based on 168k images. All these images can be generated, have signals inserted, and be reconstructed by following the instructions provided in Demo 1, Demo 2, and Demo 3, respectively.

Getting input-target training labels: For a given acceleration factor, only signal intensity–signal length combinations for which the reader achieved 100% 2AFC accuracy, as shown in the 2AFC table in 6 of Demo 2, were included in the DLMO training set for that acceleration factor. This criterion was used to avoid training DLMO on signal conditions that are formally labeled as singlets or doublets at insertion but are not reliably discriminable by human readers under the corresponding imaging and reconstruction conditions.

DLMO training components:

- Argument parsing and setup
- Data loading (training and validation)
- Model initialization and optimization setup
- Training and tuning
- Checkpoint saving and logging

Usage (without horovod build):

```
1 python dlmo_train.py [-h] [--acceleration ACCELERATION] [--train-data-path
2     TRAIN_DATA_PATH] [--val-data-path VAL_DATA_PATH] [--output-path
3     OUTPUT_PATH] [--pretrained-model-path PRETRAINED_MODEL_PATH]
4     [--pretrained-model-checkpoint-format
5     PRETRAINED_MODEL_CHECKPOINT_FORMAT]
6     [--pretrained-model-epoch PRETRAINED_MODEL_EPOCH] [--nepochs NEPOCHS]
7     --batch-size BATCH_SIZE [--val-batch-size VAL_BATCH_SIZE] [--wd WD]
8     [--shuffle_patches] [--fp16-allreduce] [--checkpoint-format
9     CHECKPOINT_FORMAT] [--save-log-ckpts] [--log-file-format
10    LOG_FILE_FORMAT]
11 Train the DLMO.
12
13 optional arguments:
14 -h, --help  show this help message and exit
15 --acceleration          Acceleration factor ([1,2,4,6,8,10,12]).
16 --train-data-path       Path to reconstructed MR images (with signals)
17                         for training DLMO.
18 --val-data-path         Path to few samples of reconstructed MR images
19                         (with signals) for tuning DLMO.
```

20	<code>--output-path</code>	Path to save checkpoints and log files
21	<code>--pretrained-model-path</code>	Transferred learning based on a previous trained model (provide path).
22	<code>--pretrained-model-checkpoint-format</code>	checkpoint file format
24	<code>--pretrained-model-epoch</code>	Transferred learning based on a previous trained model (provide epoch).
25		
26	<code>--nepochs</code>	number of epochs to train
27	<code>--batch-size</code>	Training Batch size.
28	<code>--val-batch-size</code>	input batch size for the validation/tuning data.
29	<code>--wd</code>	weight decay a.k.a regularization on weights
30	<code>--shuffle_patches</code>	Shuffles train/validation patch pairs at DataLoader level.
31		
32	<code>--fp16-allreduce</code>	Retained for CLI compatibility; ignored without Horovod.
33		
34	<code>--checkpoint-format</code>	checkpoint file format
35	<code>--save-log-ckpts</code>	saves log writer and checkpoints
36	<code>--log-file-format</code>	log file format

Training the base model at acceleration=1:

```

1 # conda activate dlmo # WITHOUT horovod build
2 ACC=1
3 NGPUS=1
4 NEPOCH=120 #dummy value
5 TRAIN_DATA_PATH=../demo3/rsos_rec/test_acc4_at_acc1_rsos.hdf5
6 VAL_DATA_PATH=../demo3/rsos_rec/test_acc4_at_acc1_rsos.hdf5
7 OUTPUT_FLD_PATH=trained_model/mri_cnn_dlmo_acc_
8
9 time python dlmo_train.py \
10 --acceleration $ACC \
11 --nepochs $NEPOCH \
12 --train-data-path $TRAIN_DATA_PATH \
13 --val-data-path $VAL_DATA_PATH \
14 --output-path $OUTPUT_FLD_PATH \
15 --batch-size 20 \
16 --val-batch-size 20 \
17 --shuffle_patches \
18 --save-log-ckpts \
19 --log-file-format log_${NGPUS}_gpus.hdf5

```

Training models with transfer learning on accelerated reconstruction:

```

1 # conda activate dlmo # WITHOUT horovod build
2 PRETRAIN_EPOCH=170
3 ACC=4
4 NGPUS=1
5 NEPOCH=35 #dummy value
6
7 PRETRAIN_PATH=../demo5/DLMO_test/trained_model/mri_cnn_dlmo_acc_1_hvd/
8 TRAIN_DATA_PATH=../demo3/rsos_rec/test_acc4_rsos.hdf5
9 VAL_DATA_PATH=../demo3/rsos_rec/test_acc4_rsos.hdf5
10 OUTPUT_FLD_PATH=trained_model/mri_cnn_dlmo_acc_
11
12 time python dlmo_train.py \

```

```

13 --acceleration $ACC \
14 --nepochs $NEPOCH \
15 --train-data-path $TRAIN_DATA_PATH \
16 --val-data-path $VAL_DATA_PATH \
17 --output-path $OUTPUT_FLD_PATH \
18 --batch-size 20 \
19 --val-batch-size 20 \
20 --shuffle_patches \
21 --pretrained-model-path ${PRETRAIN_PATH} \
22 --pretrained-model-epoch ${PRETRAIN_EPOCH} \
23 --save-log-ckpts \
24 --log-file-format log_${NGPUS}_gpus.hdf5

```

3.5 Demo 5: DLMO testing output

This implementation loads pretrained weights and DDPM-generated objects (SOMs) provided in this repository (<https://github.com/DIDSR/DLMO>) and walks through previous demos (Demo 2 and Demo 3) to demonstrate the application of the DLMO observer model for obtaining discrimination-task outputs across different MR reconstruction methods (in a use-case study with an acceleration factor of 4). It consists of five parts that should be run in order:

1. Insert the singlet and doublet signals corresponding to an acceleration factor of 4 and store the resulting MR images in an HDF5 file as follows, (also detailed in ??):

```

1 cd src/demo2
2 python signal_insertion_test.py 4 0.7 '4,5,6,7,8'

```

2. Read the HDF5 file from Step 1 and perform rSOS-based conventional reconstruction at both the accelerated rate (serving as the baseline lower bound) and the fully sampled rate (serving as the reference upper bound).

```

1 cd src/demo3
2 python rsos_ddpm_test.py 4

```

3. Run an AI-based reconstruction, such as the included U-Net pretrained weights, on the accelerated images from step 2.

```

1 cd src/demo5/AI_rec
2 python DL_denoiser_eval.py --task rayleigh --test-path ../../demo3/rsos_rec
3 --acceleration 4 --model_name unet --num-channels 1 --batch-size 10
4 --pretrained-model-path trained_model

```

4. Apply model observers to the three HDF5 files from Steps 1-3 for the three different reconstructions.

```

1 # DLMO applied on accelerated rSOS(4x) reconstruction
2 cd ../DLMO_test
3 python dlmo_test.py --task rayleigh --test-path ../../demo3/rsos_rec --acceleration
4 4 --batch-size 10 --pretrained-model-path trained_model/mri_cnn_dlmo_acc_4_hvd --
pretrained-model-epoch 50

```

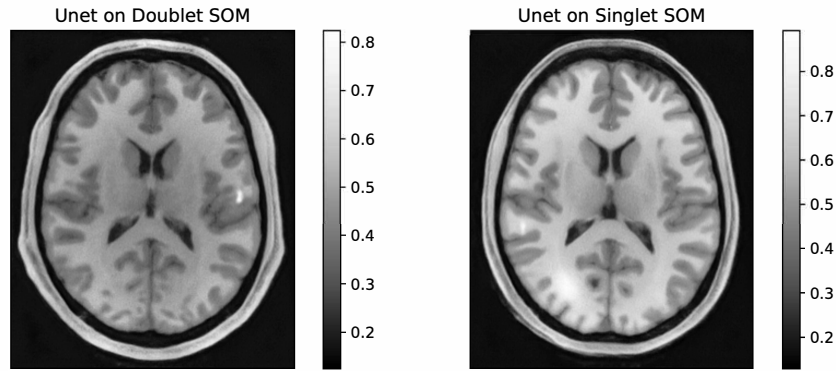


Figure 12: U-Net-based reconstruction examples with signals at an acceleration factor of 1.

```

1 # DLMO applied on accelerated U-Net(4x) reconstruction
2 python dlmo_test.py --task rayleigh --test-path ../AI_rec/ai_rec --cnn-denoiser-name
  unet --acceleration 4 --batch-size 10 --pretrained-model-path trained_model/
  mri_cnn_dlmo_acc_4_unet_hvd --pretrained-model-epoch 50

1 # DLMO applied on rSOS(1x) reconstruction
2 python dlmo_test.py --task rayleigh --test-path ../../demo3/rsos_rec --acceleration
  1 --batch-size 10 --pretrained-model-path trained_model/mri_cnn_dlmo_acc_1_hvd --
  pretrained-model-epoch 170

```

5. Process the DLMO discrimination-based outputs from the three reconstructions above and compute PC (AUC) by signal length (4–8 mm) to generate a summary plot, as shown in fig. 4 of our DLMO paper [1]:

```

1 python step4_eval.py --dlmo-eval-on-ref './dlmo_discrimination/acc1/preds_rsos.npy'
  --dlmo-eval-on-acc-rsos './dlmo_discrimination/acc4/preds_rsos.npy' --dlmo-eval-
  on-acc-ai-rec './dlmo_discrimination/acc4/preds_unet.npy' --l-list-file '../../
  demo3/rsos_rec/test_acc4_at_acc1_rsos.hdf5' --recon-method-str 'rSOS_1x,rSOS_4x,U
  -Net_4x' --signal-length-arr 4 5 6 7 8

```

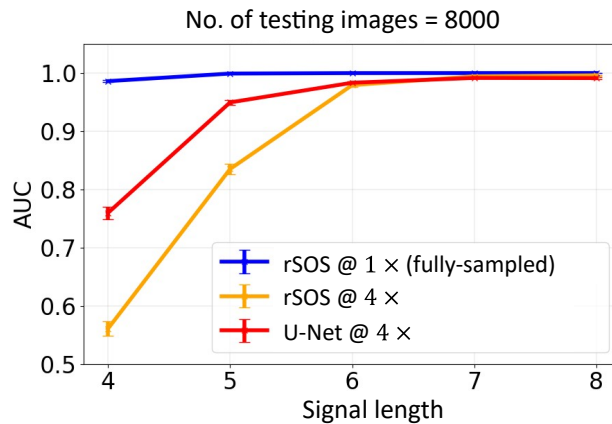


Figure 13: AUC (or percent correct) values obtained by DLMO with rSOS and U-Net reconstructions at acceleration factors of $4\times$ with 4,000 singlet and 4,000 doublet samples.

3.6 Demo 6: Statistical analysis

Statistical analysis includes two example R scripts: 1) sample size determination using a pilot data and 2) statistical analysis for a pivotal study. These R scripts have been tested with versions 4.3.1 and 4.4.1. Pre-installation of the iMRMC application is NOT recommended for the use of these scripts.

1. **Sample-size determination using pilot study:** This implementation performs sample size determination for a split-plot design, as detailed in Section IV in the supplementary material of our [DLMO paper](#) [1]. To run the script, simply execute `sample_size_est_BDG.R`. To use your own pilot data, please replace `pilot_data.csv` with your data following the same format, and update proportion correct by DLMO and its variance in the `sample_size_est_BDG.R`.

Additional details on sample size estimation using study designs other than split-plot, such as fully crossed, flat without overlap, and flat with overlap, fixed number of readers are provided in [sizing_2AFC.pdf](#)

2. **Pivotal study:** This script conducts a similarity test to investigate whether DLMO performs similarly to human readers within a predefined margin of 0.1 proportion correct (i.e., for the positively valued AUC difference, is the upper bound < 0.1 ?). To run the script, simply execute `similarity_test.R`. To update `similarity_test.R` for your own project, please update the DLMO reading results section in `similarity_test.R`, and provide reading scores in the `reading_scores` folder following the same format. Below are some additional guide:

- a. **2AFC study design:** The pivotal human-observer validation study follows a 2AFC design. In each trial, the reader is shown one singlet image and one doublet image reconstructed under the same imaging condition. The reader's task is to select the image containing the doublet signal. The primary human-observer figure of merit is proportion correct (PC). Under the 2AFC framework, PC is interpreted as the AUC corresponding to a rating-based study on the same binary discrimination task.

In the study described in our DLMO paper [1], the objective was to determine whether the human-label-trained DLMO was statistically similar to human readers on the Rayleigh discrimination task. The pre-specified similarity margin was 0.1 PC. The pivotal study used 640 singlet-doublet pairs and four trained non-physician readers in a split-plot design. Each reader evaluated 160 independent pairs for each reconstruction-method and acceleration-factor condition. The evaluated reconstruction methods were rSOS and U-Net, and the evaluated acceleration factors were 4x and 8x. Therefore, each reader completed four 2AFC studies: (i) rSOS, 4x acceleration, (ii) U-Net, 4x acceleration, (iii) rSOS, 8x acceleration, and (iv) U-Net, 8x acceleration.

For each condition, each image pair should contain a singlet and doublet image with the same signal intensity and signal length.

- b. **Preparing 2AFC image pairs:** For each acceleration factor, reconstruction method, signal intensity, and signal length:
 - i. Generate singlet and doublet image pairs. The two images in each pair (corresponding to a single 2AFC case) should have the same signal intensity, signal length,

acceleration factor, and reconstruction method. They should differ only in signal type. In our 2AFC experiment, the image pairs also differed in their backgrounds, as shown in fig. 14 below. It will also work if the same background is used when generating the singlet and doublet image pair for each 2AFC case.

- ii. Randomize the left/right or first/second display order for singlet and doublet images.
- iii. Record the correct answer for each pair, i.e., which displayed image contains the doublet signal.
- iv. Assign each pair to a single reader when using a split-plot design. In our DLMO paper, cases were non-overlapping across the four readers.
- v. Blind the reader to the correct answer, reconstruction method, and any information that could reveal whether an image contains a singlet or doublet signal.



Figure 14: An example 2AFC trial in which an image pair is displayed to a reader. One image contains a singlet signal and the other contains a doublet signal. The reader is asked to select the image containing the doublet signal. The reader may also provide a confidence score if the reader-study interface supports it.

The study may be implemented using any 2AFC reader-study interface. In our DLMO paper, we used a web-based 2AFC application, <https://rad-apps.mir.wustl.edu/twoafc>. The present repository only stores the resulting reader scores and performs the iMRMC-based similarity analysis; it does not provide the reader-study web interface.

c. **Reader-score file format:** Human-reader results should be saved under

```
1 reading_scores/<reconstruction_method>/R<reader_number>_acc<acceleration_factor>_L<signal_length>.csv
```

For example:

```
1 reading_scores/rsos/R1_acc4_L5.csv
2 reading_scores/unet/R4_acc8_L8.csv
```

An example row in a reader-scored .csv file:

Table 1: Description of columns in a .csv file from a 2AFC reader study.

Column	Description
User	Reader identifier, e.g., R1, R2, R3, or R4.
ImageID	Case identifier within a given signal-length subset. In the current script, these IDs are internally offset by signal length so that cases with different signal lengths have unique identifiers.
Correctness	Binary 2AFC result. A value of 1 indicates that the reader correctly selected the doublet image, and 0 otherwise.
Confidence	Optional reader confidence score. This column is retained for documentation purposes but is not used by <code>similarity_test.R</code> .
Comments	Optional comments provided by the reader. This column is not used by <code>similarity_test.R</code> .

```
1 User, ImageID, Correctness, Confidence, Comments
2 R1, 3, 1, 4,
```

- d. **Running the similarity analysis:** Before running `similarity_test.R`, choose the acceleration factor and reconstruction method near the top of the script:

```
1 acc <- '8'
2 rec_method <- 'unet'
```

Valid values in the provided example data are:

- acc: '4' or '8'
- rec_method: 'rsos' or 'unet'

The script then:

- Reads the corresponding human-reader CSV files from `reading_scores/<rec_method>/.`
- Curates the reader data using `createIMRMC2AFCdf()`.
- Estimates human-observer PC/AUC and variance using `doIMRMC()`.
- Loads the corresponding DLMO AUC and variance from the DLMO reading results block.
- Computes the DLMO-human difference and its confidence interval.

When adapting this script to a new reconstruction method, the DLMO should be appropriately retrained or retuned for that method. The AUC and variance values should be replaced with estimates derived from the DLMO test statistic outputs corresponding to the new reconstruction method and the acceleration factor under evaluation. Similarly, the 2AFC human reader-based AUC and variance values should likewise correspond to the new reconstruction method and the same acceleration factor.

- e. **Final comments for adapting the 2AFC study and DLMO testing:**

- i. Randomize display order to avoid positional bias.
- ii. Use independent case assignments when implementing a split-plot design.
- iii. If multiple reconstruction methods or acceleration factors are tested, account for multiplicity when interpreting the similarity tests. Our DLMO paper [1] used Bonferroni correction across the four reconstruction-method and acceleration-factor conditions.
- iv. For a new study, perform a pilot reader study and sample-size calculation before the pivotal validation study.

4 References

1. Yu, Z., Zeng, R., Samuelson, F., & Kc, P. (2026). Evaluating the resolution of AI-based accelerated MR reconstruction using a deep learning-based model observer. arXiv preprint arXiv:2602.22535.
2. Li, K., Li, H., Myers, K. J., & Anastasio, M. A. (2025, April). Estimating task-based performance bounds for accelerated MRI image reconstruction methods by use of learned-ideal observers. In Medical Imaging 2025: Image Perception, Observer Performance, and Technology Assessment (Vol. 13409, pp. 125-129). SPIE.
3. Ho, J., Jain, A., & Abbeel, P. (2020). Denoising diffusion probabilistic models. Advances in neural information processing systems, 33, 6840-6851.
4. "HCP Young Adult 2025," <https://www.humanconnectome.org/study/hcp-young-adult/document/hcp-young-adult-2025-release>, 2025.
5. Zhao, R., Yaman, B., Zhang, Y., Stewart, R., Dixon, A., Knoll, F., ... & Lungren, M. P. (2022). fastMRI+, clinical pathology annotations for knee and brain fully sampled magnetic resonance imaging data. Scientific Data, 9(1), 155.

5 Contact

- Regulatory Science Tools (RST) Catalog website: <https://cdrh-rst.fda.gov/>
- RST mailbox: RST_CDRH@fda.hhs.gov
- Developers: prabhat.kc@fda.hhs.gov, zitong.yu@fda.hhs.gov